

Detailed SOLID Principles Implementation

Calculator Console Application — Assignment 3

Prepared by: Ashutosh Kumar Tiwari

1. SOLID Principles Summary

Principle	Simple Meaning	Used In Project	Main OOP Concept	Benefit
SRP	One class should have one responsibility.	InputManager, DisplayManager, FileManager, HistoryManager, Calculator	Encapsulation	Easy to understand, debug, and modify.
OCP	Add new features without changing existing tested code.	ICalculatorOperation with separate operation classes	Abstraction + Polymorphism	New operations can be added safely.
LSP	Child classes should work wherever parent abstraction is expected.	AdditionOperation, SubtractionOperation, MultiplicationOperation, DivisionOperation	Inheritance/Interface + Polymorphism	Calculator can use any operation object safely.
ISP	Use small focused interfaces.	IDisplayManager, IInputManager, IFileManager, IOperationFactory, IHistoryManager	Abstraction	Classes are not forced to use unnecessary methods.
DIP	Depend on interfaces, not concrete classes.	Calculator constructor receives interfaces	Abstraction + Dependency Injection	Loose coupling and easier testing.

2. SRP — Single Responsibility Principle

Definition

A class should have only one reason to change. This means one class should focus on one responsibility only.

Where it is used

- InputManager: Handles user input and input validation only.
- DisplayManager: Handles console display and messages only.
- FileManager: Handles reading and writing data to the history file only.
- HistoryManager: Maintains calculation history only.
- Calculator: Coordinates the application workflow only.

Why this is good

- If display format changes, only DisplayManager needs changes.
- If the history file path changes, only FileManager needs changes.
- If input validation changes, only InputManager needs changes.

- This reduces side effects because one change does not affect unrelated classes.

3. OCP — Open/Closed Principle

Definition

Software classes should be open for extension but closed for modification. New behavior should be added by creating new code, not by repeatedly changing existing tested code.

Where it is used

- ICalculatorOperation defines a common operation contract.
- AdditionOperation, SubtractionOperation, MultiplicationOperation, and DivisionOperation implement the contract.
- Existing operation classes do not need to be modified for the new operation.

4. LSP — Liskov Substitution Principle

Definition

A child class or implementation should be usable wherever the parent abstraction is expected, without breaking the program behavior.

Where it is used

- All operation classes implement ICalculatorOperation.
- The Calculator can store any operation object in an ICalculatorOperation reference.
- The Calculator can call Execute() without checking whether the object is AdditionOperation, DivisionOperation, or another operation.
- Each operation follows the same expected behavior: receive two numbers and return a calculation result.

5. ISP — Interface Segregation Principle

Definition

Clients should not be forced to depend on methods they do not use. Interfaces should be small and focused.

Where it is used

- IDisplayManager contains display-related methods only.
- IInputManager contains input-related methods only.

- IFileManager contains file-related methods only.
- IHistoryManager contains history-related methods only.
- IOperationFactory contains operation creation methods only.

6. DIP — Dependency Inversion Principle

Definition

High-level classes should not depend directly on low-level concrete classes. Both should depend on abstractions.

Where it is used

- Calculator depends on IInputManager instead of direct InputManager.
- Calculator depends on IDisplayManager instead of direct DisplayManager.
- Calculator depends on IFileManager and IHistoryManager instead of directly handling storage and history.
- Calculator depends on IOperationFactory instead of directly creating operation objects everywhere.